

?? 目录

- scNotebook SSH 远程隧道集成 — 完整开发方案
- 一、功能总览
- 二、技术选型
- 2.1 SSH 库
- 2.2 依赖获取方式
- 2.3 目录规划
- 三、架构设计
- 3.1 整体架构图
- 3.2 数据流
- 四、完整待做清单（按执行顺序）
- Phase 1：基础设施（~3个任务）
- 任务 1.1：引入 libssh2 依赖
- 任务 1.2：SshSession 封装（核心）
- 任务 1.3：ConnectionManager 单例
- Phase 2：远程终端（~2个任务）
- 任务 2.1：SshChannel 封装（PTY 终端）
- 任务 2.2：集成到 EmbeddedTerminal（复用 TerminalView）
- Phase 3：SFTP 文件操作（~3个任务）
- 任务 3.1：SftpClient 封装
- 任务 3.2：远程文件树 UI
- 任务 3.3：远程文件编辑集成
- Phase 4：连接管理与体验优化（~3个任务）
- 任务 4.1：连接管理对话框
- 任务 4.2：设置页集成
- 任务 4.3：断线重连 + 健壮性
- Phase 5：高级功能（可选，MVP 之后）

- 五、任务依赖关系
- 六、工作量估算汇总
- 七、验收标准 (MVP 版本)
- 八、风险与缓解

scNotebook SSH 远程隧道

集成 — 完整开发方案

一、功能总览

将 scNotebook 从本地编辑器升级为 本地+远程双模式编

辑器： - 通过 SSH 连接虚拟机/远程服务器 - 远程终端

(SSH PTY) — 复用现有 TerminalView - SFTP 文件浏览/
编辑/保存 - 连接管理 + 断线重连

二、技术选型

2.1 SSH 库

方案	优点	缺点	推荐度
libssh2 (C 库)	成熟稳定、文档全、Windows	需手动封装Qt接口	★★★ ★★

	支持好		
QSSH (Qt 封装)	Qt原生API	维护不活跃、bug多	★ ★ ★
libssh (C 库)	功能全面	API复杂、Windows 编译麻烦	★ ★ ★
自建socket 实现	零依赖	工作量巨大、安全风险高	★

最终选择：libssh2 + 自行封装 Qt 接口层

理由： - Windows 下 MinGW/GCC 编译友好（有预编译包） - 异步非阻塞 API，适合 GUI 应用 - 支持 SSH2 全部特性：密钥认证、端口转发、SFTP

2.2 依赖获取方式

libssh2

2.3 目录规划

libssh2

三、架构设计

3.1 整体架构图

libssh2

3.2 数据流

libssh2



四、完整待做清单（按执行顺序）

Phase 1：基础设施（~3个任务）

任务 1.1：引入 libssh2 依赖

- 下载 libssh2 Windows 预编译包（或 vcpkg install）
 - 放入 `libssh2` 目录
 - 修改 `CMakeLists.txt`：添加 include 路径 + link 库 + DLL 部署
 - 创建 `libssh2` 基础骨架
 - 编写最小测试：初始化 libssh2 + 清理，验证编译通过
- 预估工作量：**0.5天**产出物：**可编译的空壳 SSH 模块

任务 1.2：SshSession 封装（核心）

创建 `SshSession`



具体实现要求：

- [] TCP socket 连接（Winsock2 on Windows）
- [] libssh2sessionhandshake() 握手
- [] 密码认证：libssh2userauthpassword()
- [] 密钥认证：libssh2userauthpublickey_fromfile()
- [] host key 验证（首次连接指纹确认）
- [] 异步非阻塞模式（libssh2setblocking(session, 0)）
- [] 心跳保活（keepalive 每30秒）
- [] 断线检测与自动清理

预估工作量：1天 **产出物：**可建立/断开 SSH 连接

任务 1.3：ConnectionManager 单例

创建 `ConnectionManager`



具体实现要求：

- [] JSON 格式持久化（复用现有

QJsonDocument 能力) - [] 密码 AES 加密存储 (或使用系统凭据管理器) - [] 多连接并行管理 - [] 连接状态枚举: Disconnected / Connecting / Connected / Error - [] 信号驱动 UI 更新 - **预估工作量**: 0.5天 - **产出物**: 连接配置的增删改查 + 多会话管理

Phase 2: 远程终端 (~2个任务)

任务 2.1: SshChannel 封装 (PTY 终端)

创建



具体实现要求: - [] libssh2channelopensession() + requestpty("xterm-256color") - [] libssh2channelshell() 或 libssh2channelexec() - [] QTimer 10ms 轮询: libssh2channelread() 非阻塞读取 - [] stdout/stderr 分离读取 - [] 窗口大小变化信号: SIGWINCH → libssh2channelrequestptysize() - [] Ctrl+C 发送: write("\x03") - [] EOF 检测: libssh2channeleof() - **预估工作量**: 1天 - **产出物**: 可用的 SSH PTY 通道

任务 2.2: 集成到 EmbeddedTerminal (复用 TerminalView)

修改现有文件:

EmbeddedTerminal.h/cpp 变更: - [] 新增 方法 - [] TerminalBackend 改为接口 (ITerminalBackend), 新增 SshTerminalBackend 实现 - [] 或者更简单: 让 TerminalBackend 同时支持 QProcess 和 SshChannel (策略模式)



widget.cpp 变更: - [] 侧边栏新增"远程"按钮 (在活动栏底部) - [] 点击后弹出连接列表 / 新建连接对话框 - [] 连

接成功后自动打开远程终端标签 - [] 远程终端标签显示为 "hostname: user@192.168.x.x" - **预估工作量**: 0.5天 - **产出物**: 可在 UI 中打开并使用 SSH 远程终端

Phase 3: SFTP 文件操作 (~3个任务)

任务 3.1: SftpClient 封装

创建



具体实现要求: - [] libssh2sftpinit() 初始化 - [] libssh2sftpopendir() + readdir() 目录遍历 - [] libssh2sftpopen() + read/write 文件传输 - [] 进度回调 (每4KB触发一次信号) - [] 大文件分块传输 - [] 错误处理: 权限不足/空间不足/断连恢复 - **预估工作量**: 1天 - **产出物**: 完整的 SFTP 客户端能力

任务 3.2: 远程文件树 UI

创建

UI 设计: `



具体实现要求: - [] 继承 QTreeWidget (与本地 SideBar 文件树风格一致) - [] 异步加载目录 (避免 UI 卡顿): 点击展开 → 后台 SFTP → 回调填充 - [] 图标区分: 文件夹/文件/可执行/符号链接 - [] 右键菜单: 打开/下载/上传/删除/重命名/新建 - [] 双击文件 → 触发 downloadToTemp

→ 打开编辑器标签 - [] 标签标题显示远程路径前缀 - []
加载中显示 spinner 动画 - [] 断连时显示警告提示 - **预估**
工作量: 1天 - **产出物**: 可视化的远程文件浏览器

任务 3.3：远程文件编辑集成

修改现有文件:

EditorTabBar 变更:

- [] 标签数据结构增加字段: `isRemote`
- [] 远程标签图标加一个小云朵/链式图标标识
- [] 保存时检测 `isRemote` → 调用 `SFTP upload`

MyTextEdit/FileOperator 变更: - [] 新增

SftpFileOperator 实现 IFileOperator 接口 - [] openFile(): SFTP download 到临时文件 → 打开 - [] saveFile(): 保存到临时文件 → SFTP upload 回远程 - [] 关闭标签时清理临时文件

widget.cpp 变更: - [] 侧边栏切换: 本地/远程文件树 (QStackedWidget 切换) - [] 远程文件保存进度条显示 (状态栏或覆盖层) - [] 编辑远程文件时的状态栏显示: "已连接 VM-Ubuntu | 远程文件" - **预估工作量:** 1天 - **产出物:** 完整的远程文件编辑 workflow

Phase 4: 连接管理与体验优化 (~3个任务)

任务 4.1：连接管理对话框

创建

UI 设计：SSH 连接管理

```
| | | [+ 新建连接]
| | | |
| | | ● VM-Ubuntu user@192.168.56.101 | | | [连接] [编辑] [断开] [删除] | | |
| | | ○ CentOS root@192.168.56.102 | | | [连接]
```

[编辑] [] [删除] | | |

| | | ● AWS-EC2 ec2-user@52.xx.xx | | | [连接]
[编辑] [断开] [删除] | | |

| | | | ☒ 启动时自动连接上次使用的 | | | | [取消]
[确定] |

新建/编辑连接子对话框：` └─ 编辑连接

| | | 名称:
[VM-Ubuntu_] | | 主机: [192.168.56.101] | | 端口: [22_] |
| | 用户名: [user_] | | | 认证方式: (●) 密码 () 密钥 ()
Agent | | | 密码: [·····] ◎ | ← 显示/隐藏 | | |
或 密钥路径: [Browse...] | | 密钥密码: [] | | | | ☒ 保
存密码 | | | | [测试连接] [取消] [确定] |

具体实现要求： - [] QDialog 子类，暗色主题样式 - [] 表
单验证（主机必填、端口范围 1-65535） - [] "测试连接"
按钮 → 尝试连接 → 显示成功/失败提示 - [] 密码输入框带
显示/隐藏切换 - [] 密钥文件选择器（QFileDialog） - []
连接状态指示灯（绿/黄/红） - **预估工作量**：1天 - **产出**
物：完整的连接管理 UI

任务 4.2：设置页集成

修改 SettingsPage：

- 导航列表新增"远程"分类（第6个）
- 默认连接选择
- 终端默认尺寸（列数/行数）
- SFTP 传输缓冲区大小
- 断线重连间隔（秒）
- 超时时间（秒） **预估工作量**：0.5天

任务 4.3：断线重连 + 健壮性

- SSH 断连检测 (poll 返回错误 / socket 异常)
- 自动重连机制 (指数退避: 1s → 2s → 4s → 8s → 最大30s)
- 重连中 UI 状态提示 (终端区显示"正在重连...")
- 重连成功后恢复 SFTP 文件树
- 操作中断保护 (上传/下载中途断连 → 暂停 → 重连后继续传)
- 网络异常友好提示 (不可达/拒绝/超时/认证失败) **预估工作量: 1天**

Phase 5: 高级功能 (可选, MVP 之后)

#	功能	说明	优先级
5.1	端口转发	SSH -L/-R 端口映射, 调试远程服务	低
5.2	SCP 批量传输	整文件夹上传/下载	低
5.3	跳板机/Jump Host	A → B → C 多跳连接	低
5.4	SSH Config 解析	读取 ~/.ssh/config 自动填充	中
5.5	文件同步	本地↔远程双向同步 (类似 rsync)	低
5.6	终端分屏	远程终端也支持水平分割	低

五、任务依赖关系



关键路径（最短完成时间）： `1.1 → 1.2 → (2.1 → 2.2) || (3.1 → 3.2 → 3.3) → 4.1` **MVP 最小可用版本：** Phase 1 + Phase 2（约 3 天工作量）

六、工作量估算汇总

Phase	任务数	预估工时	累计
Phase 1: 基础设施	3	2天	2天
Phase 2: 远程终端	2	1.5天	3.5天
Phase 3: SFTP 文件	3	3天	6.5天
Phase 4: 连接管理	3	2.5天	9天
Phase 5: 高级功能	6	可选	—
总计 (MVP)	8 个核心任务	~6.5天	—

七、验收标准（MVP 版本）

- 可以添加/编辑/删除 SSH 连接配置
- 可以通过密码或密钥认证连接到虚拟机
- 连接后在底部面板打开远程终端，可以正常执行命令
- 远程终端 ANSI 输出正确渲染（颜色/光标）
- 左侧显示远程文件树，可以浏览目录
- 双击远程文件可以在编辑器中打开并编辑
- Ctrl+S 保存远程文件（上传回服务器）
- 断线时有明确提示，重连后可恢复
- 切换主题时远程相关 UI 样式跟随变化

八、风险与缓解

风险	概率	影响	缓解措施
libssh2 Windows 编译 问题	中	高	使用预编译包 / vcpkg
SSH 连接被防 火墙阻挡	中	中	设置页添加代理选项
大文件传输内 存溢出	低	高	分块流式传输（4KB块）
PTY 终端兼容 性差	低	中	fallback 到 exec mode
密码安全性	中	中	Windows DPAPI 加密存储

Generated by scNotebook | 2026-06-18 15:16:18